

第 1 讲 课程概览与计算平台

对应讲义 CM5235_Lecture_01.pdf (42 页) · 逐页翻译 + 零基础讲解 + 拓展

先读这段

这一讲不考公式，但它决定你后面三讲能不能“读懂算出来的东西”。第 1 讲讲的是“计算化学到底在算什么、用什么机器算、为什么算得慢”。这三个问题的答案，会在第 2–4 讲反复以公式的形式回来找你。比如这里说“矩阵对角化是 $O(n^3)$ 且难并行”，到第 3 讲就变成“SCF 每一轮都要对角化 Fock 矩阵，所以基组一大就慢得要死”。所以别跳过。

p1 封面：Applied Computational Chemistry

原文翻译

应用计算化学 (Applied Computational Chemistry)，主讲：Livia Ungur，课程代码 CM5235。

讲解 · 课名里的 “Applied” 是什么意思

“Applied (应用)”这个词是有信息量的。计算化学大致分两支：

- **Method development (方法开发)**：发明新的近似、新的泛函、新的算法。做这个要写代码、推公式。
- **Applied / production (应用)**：用现成的软件包 (Gaussian、ORCA、VASP...) 去回答一个真实的化学问题。

这门课是后者。所以理论部分的目标不是让你能自己写一个量化程序，而是让你在看到 `#p opt b3lyp/6-31G(d)` 这一行输入时，知道每一个关键词背后省略了什么近似、这个近似在什么情况下会崩。

p2 课程计划：理论课

原文翻译

- “计算化学”总览及其与其他研究领域的关系
- 原子级与粗粒化力场方法：背景与方法局限
- 量子力学原理与基础知识：算符、本征值方程、量子化、由氢原子薛定谔方程得到的原子轨道
- Hartree–Fock 理论与 Roothaan–Hall 方程
- 波函数分析：原子电荷、电子密度、轨道定域化等
- 电子相关方法：MP2、CCSD 等
- 分子动力学：DFT 与半经验计算

讲解 · 这七条其实是一条主线

初学者看这个大纲会觉得是七个互不相干的题目。其实它是一条**精度阶梯**，从最粗糙爬到最精确，每爬一级就把上一级的一个近似去掉：

层级	做了什么近似	典型体系规模
力场 (MM)	完全放弃电子，把分子当成弹簧连起来的小球	10^5 – 10^7 原子 (蛋白质、聚合物)
半经验	保留电子，但双电子积分大部分靠参数拟合	10^3 原子
Hartree–Fock	保留电子，双电子积分全算，但用 平均场 ：每个电子只感受到其他电子的"平均云"	10^2 原子
DFT	与 HF 同形式，但用近似泛函把"相关能"补回来一部分	10^2 – 10^3 原子
MP2 / CCSD	显式地把电子的 瞬时躲避 (相关运动) 算进来	20–100 原子
Full CI	不做任何波函数近似 (只剩基组不完备)	3–4 个原子

整门课的核心情节就是：**"电子相关 (electron correlation)" 这四个字**。HF 把它丢掉了，后面所有方法都是在想办法把它捡回来，代价是计算量爆炸。第 4 讲整讲都在讲这件事。

拓展 · "粗粒化 (Coarse-Grained)" 是什么

全原子力场里，一个 CH_2 是 3 个粒子 (1 个 C + 2 个 H)。粗粒化力场把整个 CH_2 当成**1 个粒子 (bead, 珠子)**，参数重新拟合。粒子数少了 3 倍，但更关键的是：粒子重了、相互作用软了，积分步长可以从 1 fs 拉到 20–40 fs，于是能模拟的时间尺度直接跳两三个数量级 (从 ns 到 μs –ms)。代表是 MARTINI 力场，做膜蛋白、脂质体、聚合物自组装的标配。代价是丢掉了化学细节——粗粒化模型里没有"键的断裂"这回事。

p3-4 课程计划：实践课

原文翻译

p3：实践课包含一些直接的任务，用来熟悉 Linux 操作系统、脚本、输入文件与作业的准备和提交。我们将熟悉并使用以下量子化学软件包：Gaussian、ORCA、OpenMOLCAS 和 VASP。Gaussian 与 ORCA 将用于：几何优化与振动频率计算、轨道与电子密度/自旋密度的可视化、磁交换 (BS-DFT)、以及通过 TD-DFT 计算激发态。

p4：我们将用 OpenMOLCAS 做多组态方法 RASSCF/CASPT2/RASSI，计算过渡金属与镧系化合物的电子激发态。最后，用 VASP 研究固态周期性材料、二维周期层、以及分子在表面的沉积。上述每个软件包都有一整套结果处理工具 (用 python、bash 等写成)。

讲解 · 四个软件的分工，一句话记住

软件	擅长什么	为什么用它
Gaussian	有机分子的常规活：优化、频率、DFT、溶剂化	输入语法最宽容，配 GaussView 图形界面，上手最快
ORCA	光谱性质：NMR、EPR、零场分裂 ZFS、磁交换	对学术界免费，光谱模块业界第一
OpenMOLCAS	多组态：CASSCF / CASPT2 / RASSI	开源，处理“单个 Slater 行列式不够用”的体系（过渡金属、镧系、双自由基）
VASP	固体、周期性体系、表面吸附	平面波 + 赝势，能带、态密度、声子

拓展 · 名词解释（这些词后面会一直出现）

- **几何优化 (geometry optimization)**：在势能面上找一个梯度为零的点（能量最低点）。你给一个初始结构，程序反复挪原子直到力 ≈ 0 。
- **振动频率 (vibrational frequency)**：算能量对坐标的**二阶导数**（Hessian 矩阵），对角化后得到简正模式。
它的用途不只是模拟红外光谱——更重要的是判断你优化到的到底是极小点还是过渡态：全是正频率 = 真极小点；有 1 个虚频（程序打印成负数）= 一阶鞍点 = 过渡态。
- **BS-DFT (Broken-Symmetry DFT, 破缺对称 DFT)**：算双核金属配合物里两个未成对电子是铁磁（平行）还是反铁磁（反平行）耦合的技巧。第 3 讲 p45 会提到它和交换积分 K 的关系。
- **TD-DFT (Time-Dependent DFT, 含时密度泛函)**：算激发态能量（ $\rightarrow$ UV-Vis 光谱）最常用的方法。
- **赝势 (pseudopotential) / ECP (Effective Core Potential, 有效核势)**：把重原子的内层电子换成一个等效势函数，不再显式计算。既省钱，又顺手把相对论效应塞进去了。

p5 考核方式

原文翻译

四次作业，都是在 HPC@NUS 集群上做的实际计算练习：

- GAUSSIAN / GAUSSVIEW (占 15%)
- ORCA / Avogadro / Chemcraft (占 15%)
- OpenMOLCAS / Luscus (占 15%)
- VASP / VESTA / vaspkit / py4vasp (占 15%)

请按时提交报告，每人一个 PDF 文件。手写文件也可以——但请确保我能看懂你的字！

期中测验 (9 月 29 日周二 19:00–20:00, 占 15%)；期末考试 (占 25%)。期中与期末主要是理论与概念题，不需要做实际计算。

划重点

"mostly theoretical and conceptual questions. No practical calculations will be needed."

这句话直接定义了你的复习策略：**考的是"为什么"和"什么时候会失败"，不是"算出数字"**。典型考法是——

- "RHF 和 UHF 在 H_2 解离时给出的曲线为什么不同？"
- "为什么截断 CI (如 CISD) 不是 size-consistent, 而 CCSD 是？"
- "什么情况下必须用 CASSCF 而不能用 HF/DFT？"
- "给一个 Slater 行列式的占据图，写出它的能量表达式 (用 h 、 J 、 K)。"

最后一类是唯一需要"动笔算"的，但只用加加减减——第 3 讲 p44–45 有原题。

p6 推荐读物

原文翻译

值得读的书 (幻灯片只放了封面图)。

讲解 · 这个领域的标准书单

书	定位
Jensen, <i>Introduction to Computational Chemistry</i>	这门课的最佳配套 。覆盖面和本课大纲几乎一一对应，公式不多但把"为什么"讲透了。第 3–5 章正好是本课第 2–4 讲。

Szabo & Ostlund, <i>Modern Quantum Chemistry</i>	HF 和后 HF 的圣经，Dover 平装很便宜。所有推导都写全了。如果你想真正搞懂 Fock 算符是怎么从变分法里蹦出来的，看这本第 3 章。
Cramer, <i>Essentials of Computational Chemistry</i>	比 Jensen 更偏"实用"，讲了大量方法的实际表现和陷阱。
Levine, <i>Quantum Chemistry</i>	本科教材风格，最基础。第 2 讲的氢原子部分 (p44–55) 基本就是这本书第 6 章。讲义里 p54 那张径向波函数表格就是从 Levine 抄的 (Table 6.1)。

p7 什么是计算化学？

原文翻译

计算化学是一个（相对新的）研究领域，其目标是通过使用**计算机和粒子间相互作用的基本定律**来回答化学问题。

例子：一个典型的化学反应有很多未知量：

- 找到最合适的反应物
- 找到最合适的催化剂
- 反应条件（温度、压力、溶剂等）
- 把新化合物从反应混合物中分离出来
- 找出它的物理性质：分子结构与晶体结构、势能面、熔点、EPR 谱、UV-VIS、IR、NMR 和拉曼谱等
- 建立反应机理，获得新的理解/洞察
- 理解反应动力学，使其能放大到工业应用

示意图：A + B + 催化剂 → 产物 + C + D + E + ...

讲解 · "基本定律"具体指哪一条

这里说的 "fundamental laws of interaction between particles"，在整门课里其实**只有一条**：**库仑定律**（带电粒子之间的静电作用）。加上量子力学的运动学（薛定谔方程）和费米子的反对称性（泡利原理），整个分子的行为原则上就完全确定了。

分子体系的完整非相对论哈密顿量（第 2 讲 p77 会正式给出）

$$\hat{H} = \hat{T}_{\text{核}} + \hat{T}_{\text{电子}} + \hat{V}_{\text{核-核}} + \hat{V}_{\text{核-电}} + \hat{V}_{\text{电-电}}$$

符号	名称	物理含义
$\hat{H}$	哈密顿算符	体系的 总能量算符 。头上那个尖帽子 "^" 表示这是一个"算符" (operator, 一个作用在函数上的操作), 不是普通的数。
$\hat{T}$	动能算符	粒子运动带来的能量
$\hat{V}$	势能算符	粒子之间的静电相互作用能, 全部来自库仑定律

为什么这么简单的东西会难? **因为最后那一项 $\hat{V}_{\text{电-电}}$ (电子之间的排斥) 把所有电子的坐标纠缠在一起, 方程无法分离变量。**整个量子化学的历史, 就是想尽办法处理这一项的历史。第 2 讲 p61 会正式说这件事。

拓展 · 计算化学在真实科研里的三种用法

1. **解释 (interpretation)**: 实验做出来了但不知道为什么, 算一下轨道/电荷/过渡态来解释。这是目前多数论文的用法。
2. **预测 (prediction)**: 实验还没做, 先算一遍决定该不该做。要求精度高得多, 风险也高。
3. **反推 (assignment)**: 谱图上有个峰不知道归属, 算几个候选结构的理论谱去对。NMR 化学位移、IR 频率、UV-Vis 跃迁都常这么干。

p8-9 领域中的重要人物与诺贝尔奖

原文翻译 (p9 诺奖部分)

- **Walter Kohn (1998)**: "因其发展了密度泛函理论"
- **John Pople (1998)**: "因其发展了量子化学中的计算方法"
- **Martin Karplus、Michael Levitt、Arieh Warshel (2013)**: "因其为复杂化学体系发展了多尺度模型"

p8 还列出了 Enrico Clementi 的贡献 (1961 年开创从头算在小分子和大分子体系的应用、免费分发 Hartree-Fock 程序、首次系统估算原子和分子的相关能、1963 年开创密度泛函方法、1983 年开创并行架构 LCAP 等) 以及一长串人名: Einstein、Schrödinger、Dirac、Fock、Pople、Kohn、Hartree、Parr、Dewar、Car、Parrinello、Warshel、Mulliken、Löwdin、Helgaker、Neese、Head-Gordon、Hohenberg、Pulay 等。

讲解 · 三个诺奖对应本课的三块内容

- **Pople (1998)** → 你要用的 **Gaussian** 就是他 1970 年写的。他真正的贡献是"把量子化学变成一个人人可用的工具": 定义了 Pople 基组 (6-31G 那一套, 第 3 讲 p76)、定义了"model chemistry"这个概念 (**方法/基组 这个写法, 比如 B3LYP/6-31G(d), 就是他推广的记法**)。
- **Kohn (1998)** → DFT。Hohenberg-Kohn 定理证明了: **基态的一切性质都由电子密度 $\rho(\mathbf{r})$ 唯一决定**。这是个惊人的降维——把 $3N$ 个变量的波函数换成 3 个变量的密度函数。

- **Karplus / Levitt / Warshel (2013)** → 多尺度 QM/MM。把反应中心用量子力学算，把周围的蛋白质/溶剂用经典力场算，两者拼起来。酶催化机理研究的标准做法。

拓展 · 名字要认，因为它们会变成关键词

这个领域大量术语是人名，考试里会直接出现：

Hartree	自洽场方法（1927）；也是原子单位的能量单位（1 Hartree = 27.2114 eV = 627.5 kcal/mol）
Fock	把反对称性（泡利原理）加进 Hartree 方法 → Hartree–Fock（1930）；Fock 算符、Fock 矩阵
Slater	Slater 行列式（用行列式保证反对称）；Slater 型轨道 STO
Roothaan	把 HF 微分方程变成矩阵方程（LCAO，1951）→ Roothaan–Hall 方程
Koopmans	Koopmans 定理：轨道能 $\approx$ -电离能（第 3 讲 p47–48）
Brillouin	Brillouin 定理：单激发态不与 HF 基态混合（第 4 讲 p18、p25 关键）
Møller–Plesset	MP2/MP3/MP4，微扰论处理相关能（第 4 讲 p36）
Löwdin	正交化方法；也是“电子相关”这个定义的提出者
Mulliken	Mulliken 布居分析（算原子电荷，第 3 讲 p81 练习里用到）
Pulay	DIIS 收敛加速算法（第 3 讲 p68）；解析梯度

原文翻译 (节选)

JACS 所有时期论文中被引最多的十篇计算类文章 ([总排名]. 标题 (引用数)) :

- 3. 量子力学分子模型的开发与使用 76. AM1: 一个新的通用量子力学分子模型 (14995)
- 6. 用于蛋白质、核酸和有机分子模拟的第二代力场 (10233)
- 10. OPLS 全原子力场在有机液体构象能量与性质上的开发与测试 (8377)
- 17. 分子基态 38. MNDO 方法——近似与参数 (6727)
- 20. UFF: 一个全周期表的分子力学与分子动力学力场 (5977)
- 24. GIAO 方法在 NMR 化学位移计算中的高效实现 (5289)
- 28. 自洽分子轨道方法 21. 第一行元素的小型劈裂价基组 (4517)
- 31. 构象分析 130. MM2——使用 V1 与 V2 扭转项的烃类力场 (4257)
- 36. 天然杂化轨道 (3872)

讲解 · 这张表在说一件事: 引用数最高的不是"最深刻的理论", 而是"最好用的工具"

看这个榜单的构成:

- 4 篇力场 (AMBER、OPLS、UFF、MM2)
- 2 篇半经验方法 (AM1、MNDO)
- 1 篇基组 (6-31G, 就是 #28 那篇)
- 1 篇性质计算方法 (GIAO 算 NMR)
- 1 篇波函数分析 (NBO 天然杂化轨道)

全是"别人拿去用"的东西, 没有一篇是纯理论。这就是应用计算化学的生态: 写一个别人天天要用的工具, 比证明一个漂亮定理的引用高一个数量级。

拓展 · 榜单里几个词的意思

- **AM1 / MNDO / PM6**: 半经验方法家族。它们的共同思路是——保留 HF 的框架, 但把最贵的双电子积分要么直接扔掉 (NDDO 近似: 只保留同一原子上的), 要么用实验数据拟合的参数代替。速度比 HF 快几个数量级, 代价是只对参数化过的元素/键型可靠。第 3 讲 p62 会提到"半经验方法基于对双电子积分的部分忽略"。
- **GIAO (Gauge-Independent Atomic Orbital, 规范不变原子轨道)**: 算 NMR 屏蔽常数时, 磁场的矢势有"规范自由度", 用有限基组会导致结果依赖于你把坐标原点放哪。GIAO 让每个基函数自带一个与位置相关的相因子, 从而消除这个依赖。今天所有软件算 NMR 默认都是 GIAO。
- **NBO (Natural Bond Orbital, 天然键轨道)**: 把离域的正则分子轨道 (第 3 讲 p46、p61 会讲) 转换成化学家熟悉的"孤对电子、 σ 键、 σ^* 反键"的定域图像, 从而能定量讨论"超共轭"、"负超共轭"这类概念。

原文翻译

1925	海森堡、玻恩、约当发展矩阵力学；薛定谔发明波动力学与非相对论薛定谔方程；Heitler 与 London 发表第一个化学键计算
1927	Douglas Hartree 发表自洽场方法
1930	Vladimir Fock 提出 Hartree–Fock 理论
1947	ENIAC——第一台通用计算机建成
1950	Clemens Roothaan 发表 LCAO 理论
1951	UNIVAC——第一台商用通用计算机
1955	第一次对"大"分子 N_2 的从头算
1956	晶体管取代计算机中的真空管
1964	Hohenberg 与 Kohn 提出密度泛函理论
1970	John Pople 推出 Gaussian 软件
1971	第一款商用微处理器 (Intel 4004)
1998	Chuang (LANL) 与 Gershenfeld (MIT) 造出第一个量子处理器 (2 量子比特)
2019	Google 用可编程超导量子处理器演示"量子优越性"
2020	中科院团队宣布基于光子的量子处理器

讲解 · 注意这条时间线的节奏

把它拆成两列你会看到一个很清楚的模式：

理论就位的时间	能真正算的时间
1926 薛定谔方程	1955： 才第一次算了 N_2 (两个原子) 1970： Gaussian 发布，普通化学家才用得上
1927 Hartree SCF	
1930 Fock 反对称化	
1951 Roothaan LCAO	

理论早在 1930 年就基本完备了，卡住这个学科 25 年的是算力。Dirac 1929 年那句名言正是这个意思——大意是：物理和化学所需的基本定律已经完全知道了，困难只在于这些定律导出的方程太复杂而无法求解。

这解释了为什么第 1 讲要花一半篇幅讲硬件（p13-27）：**在这个领域，硬件不是背景，是主线。**

拓展 · Heitler–London (1927) 为什么重要

它是第一个用量子力学解释"化学键为什么存在"的计算。对 H₂，他们写的波函数是

$$\psi_{\text{HL}} = \chi_{\text{A}}(1)\chi_{\text{B}}(2) + \chi_{\text{B}}(1)\chi_{\text{A}}(2)$$

符号	名称	含义
χ_{A}	原子轨道	A 原子上的 1s 轨道（希腊字母 chi，念"卡伊"）
(1)	电子编号	括号里的数字是 电子的编号 ，代表这个函数以电子 1 的坐标为自变量

意思是"电子 1 在 A 原子、电子 2 在 B 原子"和"反过来"这两种情形的叠加——纯共价，没有离子项。这就是**价键理论 (VB)** 的源头。

而分子轨道理论 (MO) 给出的 ψ_{MO} 展开后是 50% 共价 + 50% 离子（第 4 讲 p19 会精确地推这一步）。**这两个理论的差别正是第 4 讲的核心情节：MO/HF 在解离极限下因为强行保留 50% 离子项而给出错误的能量，CI 通过混入双激发把离子项消掉，从而修复它。**所以这条 1927 年的记录不是历史花边，它是第 4 讲 p17-21 的伏笔。

p12 未来发展方向

原文翻译

1. AI / 机器学习 / 深度学习 + 计算化学 → 有可能以比常规 DFT 快 10000 倍以上的速度得到 DFT 精度的结果
→ 允许更充分地扫描相空间、快速预测新材料/现象/反应性/性质 → 加速新分子与新材料的合成与发现

2. 弱相互作用的准确描述：如范德华力、色散、长程相关

3. 分子晶体的晶体结构与声子谱的准确预测

4. 非平衡量子动力学，即同时求解电子与原子核的薛定谔方程（非 Born–Oppenheimer）

5. 面向量子处理器的计算方法及各类混合方法的发展

讲解 · 逐条说清楚它在解决什么痛点

1. 机器学习势 (MLIP / NNP)

痛点: DFT 算一个 100 原子体系的单点能要几分钟；做分子动力学要算几百万步，根本跑不动。

做法: 用几千个 DFT 算好的构型训练一个神经网络，输入是原子坐标，输出是能量和力。训练好之后，一次预测只要毫秒。代表工作：ANI、SchNet、NequIP、MACE，以及近年的通用势 M3GNet、MACE-MP-0、DPA。

本质: 它没有解薛定谔方程，它是在**拟合势能面**。所以它对训练集覆盖不到的化学环境（比如没见过的元素组合、断键中间态）会给出不可靠的结果，而且往往不会报错。

2. 弱相互作用与色散

痛点: HF 和绝大多数常规 DFT 泛函从**根本上算不出伦敦色散力**。原因是色散来自“两个分子的电子云瞬时涨落的关联”，而 HF 是平均场（没有瞬时涨落），常规 DFT 泛函是“局域/半局域”的（只看某一点及其附近的密度，看不到远处）。

后果: 不加修正的 B3LYP 算不出 π - π 堆积、算不出石墨层间结合、算不准蛋白-配体结合能。

做法: Grimme 的 **DFT-D3 / D4** 经验修正——直接加一项 $E_{\text{disp}} = -\sum C_6/R^6$ （配一个阻尼函数防止短程发散）。你在第 3 讲 p5 的 Gaussian 输出里会看到 Adding Grimme-D3(BJ) dispersion energy 这一行，说的就是这个。

3. 晶体结构预测 (CSP) 与声子

痛点: 同一个分子可以结晶成好几种晶型（多晶型 polymorphism），而不同晶型的溶解度、稳定性可以差很多——这在制药业是生死攸关的（Ritonavir 事件：药物上市后突然出现一个更稳定的晶型，导致全球召回）。而不同晶型之间的能量差常常小于 1 kJ/mol，比目前方法的误差还小。

4. 超越 Born–Oppenheimer

痛点: BO 近似（第 2 讲 p78 详解）假定电子瞬间跟上原子核。但在“**锥形交叉 (conical intersection)**”附近，**两个电子态的势能面靠得极近甚至相交，BO 彻底失效**。而光化学、荧光、光合作用、视觉的第一步（视黄醛异构化）全都发生在这种地方。

5. 量子计算

为什么量子计算机天生适合量子化学: Full CI 的计算量随体系呈阶乘增长（第 4 讲 p40 标度表里写着 $O(n!)$ ），因为要在指数大的希尔伯特空间里做线性代数。而量子比特本身就存活在指数大的希尔伯特空间里——用量子系统模拟量子系统，是 Feynman 1982 年提出量子计算机的最初动机。代表算法：VQE（变分量子本征求解器）、量子相位估计。目前受限于噪声，还只能算 H_2 、LiH 这个级别。

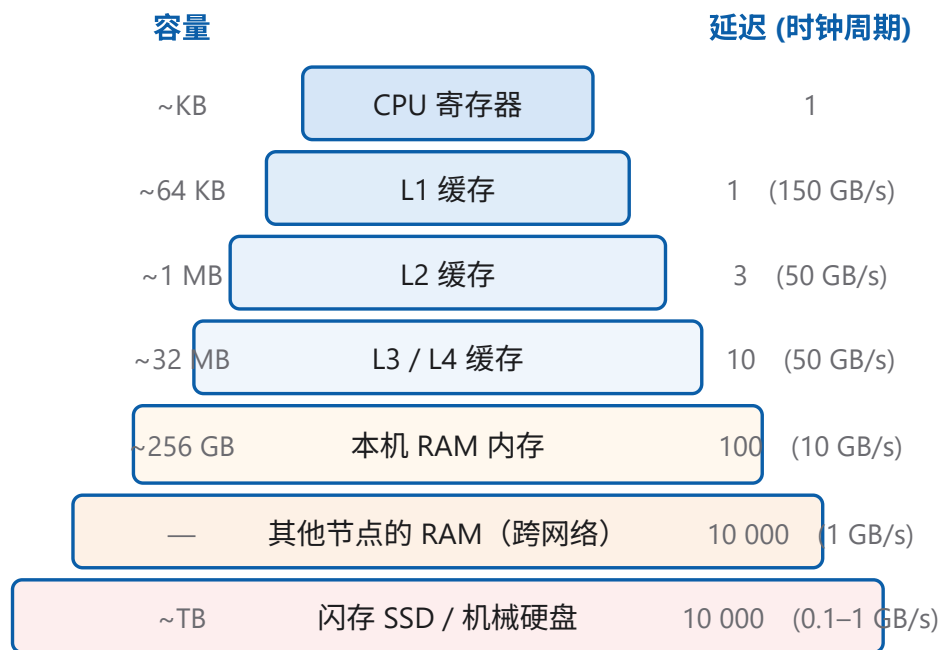
原文翻译

一块主板集成以下部件：

- CPU（一个或多个），包含：若干计算核心 (cores)、本地缓存 (cache) 内存（非常快）
- 活动 RAM 内存（访问快）
- 硬盘存储（访问慢）
- 显卡（含自己的 GPU 和显存）
- 外围部件

讲解 · 一个从慢到快的金字塔

你只要记住一个概念：**存储器是分层的，越快越小越贵**。数据必须一层层往上搬才能被 CPU 用到。



越往下：越大、越慢、越便宜

存储层级金字塔（数据取自讲义 p19 的表格）

拓展 · 为什么这跟量子化学有关

因为量子化学最核心的数据结构——**双电子积分**（第 3 讲 p62）——数量是 $N^4/8$ 量级（ N = 基函数个数）。一个中等分子， $N = 500$ ，就是约 8×10^9 个数，双精度存储要 **约 60 GB**。放不进缓存，勉强放进 RAM，一旦放不下就得往硬盘倒——速度立刻掉两个数量级。

这就是为什么会有 "direct SCF"（第 3 讲 p63）：**与其存下来再读，不如每次要用的时候现算**。因为现在 CPU 算得比内存搬得快。

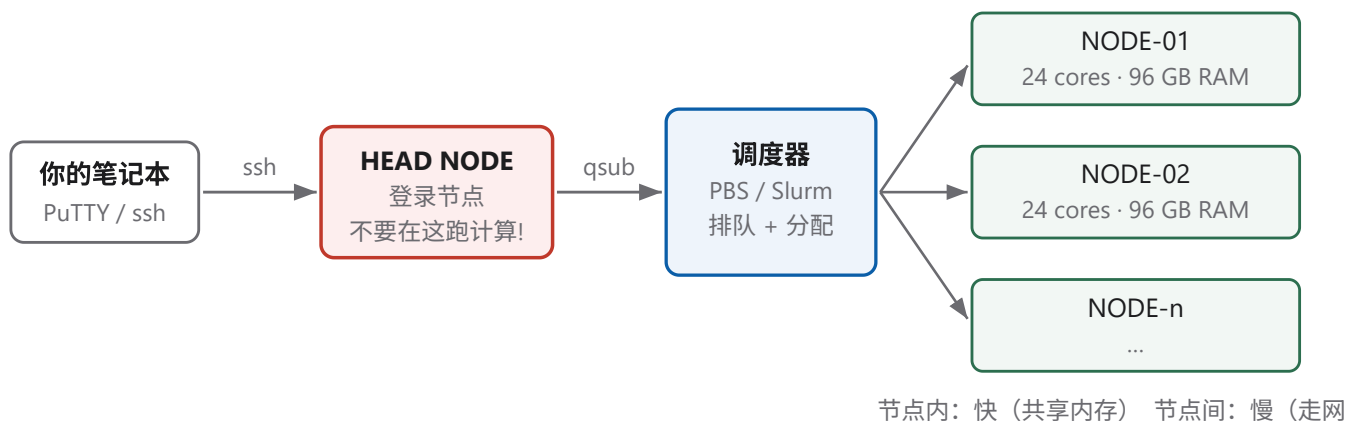
原文翻译

结构：一个 HEAD NODE（头节点/登录节点）+ 一台 Interconnect switch（互联交换机）+ 若干 NODE-01... NODE-16（计算节点）。

特点（以 NSCC 为例）：

- **异构互联 (Heterogeneous interconnect)**：节点之间的数据传输慢，同一节点内处理器（核心）之间的数据传输快
- **相当便宜**：集群的价格只比所有节点价格之和略高。可以做大规模部署
- **容错**：如果一个节点没有响应，可以单独把它关掉，不需要关停整个集群
- 通常运行 Linux 操作系统

讲解 · 头节点 vs 计算节点：新手最常犯的错



集群的工作方式：你只登录到头节点，真正的计算由调度器派发到计算节点

这是最容易被骂的错误

永远不要在头节点（登录节点）上直接跑计算。头节点是全系几十上百个人共用的，你在上面直接敲 `g16 input.gjf`，会把所有人的登录卡住，管理员可能直接封你的账号。正确做法是写一个提交脚本（p41 有模板），用 `qsub` 交给调度器。

拓展 · “异构互联”对你的实际影响

讲义反复强调节点内快、节点间慢。这直接决定了你申请资源时该怎么写：

- `select=1:ncpus=24` —— 要 1 个节点的 24 核。**对 Gaussian 来说这几乎总是正确的选择**，因为 Gaussian 的常规并行是共享内存（OpenMP/Linda），跨节点效率差（p2 的 Gaussian 表格明说“DFT 最多扩展到 32 核”）。

- `select=4:ncpus=24` —— 要 4 个节点共 96 核。只有当程序用 MPI 且通信量小时才划算（VASP、NWChem 可以）。

新手常见的心态是“多要核 = 算得快”。**实际上要多了往往更慢**：核多了通信开销上升，而且排队等待时间也变长（调度器要凑齐 4 个空闲节点比凑 1 个难得多）。

p18 共享内存超级计算机（如 IBM Blue Gene/Q）

原文翻译

商用超算（如 Blue Gene/Q）拥有专有的大规模主板，能够高效地容纳并集成大量处理器和 RAM 内存插槽。

这类超算的价格远高于同等核数的普通集群。

非常大的 RAM 内存和数千个节点的可用性，使得**在别处无法完成的独特任务**成为可能：**大矩阵对角化**。

它们运行 UNIX 操作系统，配有一套内部优化的工具。

讲解 · 为什么偏偏点名“大矩阵对角化”

这句话是给量子化学量身定制的。回想一下：**对角化一个 $n \times n$ 矩阵，计算量是 $O(n^3)$ ，但内存需求是 $O(n^2)$ ，而且它非常难并行**（p26 的表格写明 “Most matrix factorizations ... Parallel execution: HARD”）。

难并行的原因：对角化算法（如 Householder 变换 + QR 迭代）里，每一步都要用到上一步的**全部**结果，数据依赖是全局的。你可以把矩阵切开分给不同节点，但每一步都要在所有节点之间交换数据——而节点间通信慢得多。

所以：普通集群适合“能切成互不相干的小块”的任务（比如算几百万个双电子积分，每个积分互相独立）；共享内存超算适合“必须整体处理一个巨大对象”的任务（对角化一个巨大的 CI 矩阵）。

拓展 · CI 矩阵到底能有多大

第 4 讲 p15 有张表：一个体系在某基组下，CISD 的组态数是 2 556，CISDTQ 是 391 126，Full CI 到 10 重激发是 30 046 752。矩阵是这个数的平方—— 3×10^7 的平方是 9×10^{14} 个元素，双精度要好几个 PB。所以 Full CI 从来不显式存矩阵，而是用 **Davidson 迭代法**：只求最低的几个本征值，只需要能计算“矩阵乘向量”而不需要存整个矩阵。

原文翻译

延迟 (Latency)：从 CPU 发出数据请求，到数据**开始**到达所需的时间。

带宽 (Bandwidth)：数据到达的速度（速率）。

位置	延迟	带宽
CPU 寄存器	1 时钟周期	—
L1 缓存	1 时钟周期	150 GB/s
L2 缓存	3 时钟周期	50 GB/s
L3/L4 缓存	10 时钟周期	50 GB/s
本机 RAM	100 时钟周期	10 GB/s
NUMA RAM	150 时钟周期	10 GB/s
其他节点的 RAM	10 000 时钟周期	1 GB/s
闪存	10 000 时钟周期	1 GB/s
磁存储	10 000 时钟周期	0.1 GB/s

较小的数据集和程序处理得更快，因为它们能装进更高层级的缓存（缓存更快）。**对主 RAM 的随机访问会让程序变得非常慢。**

讲解 · 延迟 vs 带宽，用一个比喻讲清楚

比喻：从新加坡往北京运货

- **延迟** = 你下单之后，第一箱货到达北京要多久（比如 3 天）。
- **带宽** = 之后每小时能到多少吨（比如 10 吨/小时）。

关键：**这两个是独立的**。一辆装满硬盘的卡车横穿大陆，**带宽极高**（几百 TB / 几天），但**延迟糟糕透顶**（几天）。反过来，一条光纤延迟只有几毫秒，但带宽可能只有 1 Gbps。

讲解 · “随机访问会让程序非常慢”是什么意思

这是这一页最实用的一句话。CPU 从 RAM 取数据时，**不是一次取一个数，而是一次取一整条 “cache line”**（通常 64 字节 = 8 个双精度数）。所以：

- **顺序访问**（`a[0]`，`a[1]`，`a[2]`...）：取一次 cache line 能用 8 次，只有 1/8 的访问真正付了 100 周期的代价。

- **随机访问** (`a[7]`, `a[913]`, `a[42]`...) : 每次都 miss, 每次都付 100 周期。**慢 8 倍以上** (实际因为预取失效, 往往慢几十倍)。

拓展 · NUMA 是什么

表里出现的 **NUMA = Non-Uniform Memory Access (非一致内存访问)**。现代双路服务器有 2 个 CPU 插槽, 每个插槽**直接连着自己的一半内存条**。CPU-0 访问自己的内存要 100 周期, 访问 CPU-1 那半边要 150 周期 (得绕道 CPU 间的总线)。

实际影响: 如果你的进程被操作系统迁移到另一个插槽, 而数据还在原来那半边内存里, 性能会莫名其妙掉一大截。HPC 上跑大作业时常用 `numactl` 或调度器的 `place=scatter/pack` 选项来把进程钉在特定插槽上, 就是为了避免这个。

p20 CPU 性能指标: GIPS 与 GFLOPS

原文翻译

CPU 常用的两个性能指标是:

- **GIPS** (每秒十亿条指令) ——处理器每秒能执行多少条机器码指令。这个指标非常依赖具体架构。
- **GFLOPS** (每秒十亿次浮点运算) ——处理器每秒能做多少次单精度或双精度乘法。**科学计算几乎总是用双精度。**

功耗对单台计算机/节点来说不太关键, 平均 0.2 kWh。能耗对大型集群是个问题。

新型 ARM 处理器 (用于手机和平板) 相比 Intel 处理器能耗显著更低 (例如新的 Apple M1)。当前排名第一的超算——日本的富岳 (Fugaku)——用的就是基于 ARM 的 A64FX 处理器。

讲解 · 为什么"科学计算几乎总是双精度"

精度	位数	有效数字	典型用途
单精度 FP32	32 位	约 7 位十进制	图形、深度学习
双精度 FP64	64 位	约 16 位十进制	量子化学

为什么量化非要双精度? **因为量化算的是"大数相减得小数"**。举个真实的例子:

一个中等有机分子的总能量 $\approx -2692.659\ 129\ 53$ Hartree (第 3 讲 p5 的真实输出)

而一个化学键的能量 ≈ 0.15 Hartree, 一个氢键 ≈ 0.008 Hartree

反应的能垒差异要求精度 ≈ 0.0016 Hartree (约 1 kcal/mol, 即"化学精度")

你要在 2692 这个量级上分辨 0.001 的差别，就需要 **约 7 位有效数字**只用来抵消，剩下的才是有用信息。单精度只有 7 位——全部吃光，一位不剩。所以必须用双精度。

拓展 · 这也是"GPU 在量化里没有想象中好用"的原因

消费级显卡（RTX 40 系等）的 FP64 性能被厂商**故意限制**到 FP32 的几十分之一，因为游戏和 AI 都只需要 FP32/FP16。真正 FP64 强的是数据中心卡（A100/H100，FP64 大约是 FP32 的 1/2）。所以你不能拿游戏显卡加速 Gaussian。

近年的进展是"混合精度"：用 FP32 甚至 FP16 算积分的**大部分贡献**，只对数值上重要的部分用 FP64。TeraChem、以及 ORCA 6 的部分模块走的是这条路。

p21

摩尔定律

原文翻译

集成电路上的晶体管数量大约每两年翻一番。

现代计算机不断增长的算力，推动着计算化学扩展到**更复杂的体系**和**更精密的理论层次**。

讲解 · 摩尔定律在这门课里的残酷含义

摩尔定律说算力每 2 年翻倍。听起来很好——但请对照第 4 讲 p40 的标度表：

方法	标度	算力翻倍后，能算的体系大多少？
HF / DFT	$O(N^4)$	$\times 2^{1/4} = +19\%$
MP2	$O(N^5)$	$\times 2^{1/5} = +15\%$
CCSD	$O(N^6)$	$\times 2^{1/6} = +12\%$
CCSD(T)	$O(N^7)$	$\times 2^{1/7} = +10\%$
Full CI	$O(N!)$	基本 +0 个原子

摩尔定律给你的算力是指数增长，但方法的开销也是高次幂增长，两者相抵后，“能算多大”只是龟速爬行。等 20 年（算力涨约 1000 倍），CCSD 能算的原子数也只涨到 3 倍。

所以这个领域真正的进步来自**算法**而不是硬件：把 $O(N^4)$ 通过 RI 近似降到 $O(N^3)$ （第 3 讲 p64）、通过局域化降到接近 $O(N)$ （线性标度方法，如 DLPNO-CCSD(T)）。**一个把标度降一阶的算法，抵得上几十年的摩尔定律。**

拓展 · 摩尔定律其实已经改道了

2005 年之后，**单核频率停止增长**（卡在 3–5 GHz，因为功耗随频率急剧上升，散热扛不住）。晶体管数还在涨，但涨出来的都用去堆核数了。后果：**你的程序如果不会并行，那从 2005 年到现在它几乎没变快。**这就是为什么这门课要花时间教你写 `#PBS -l select=1:ncpus=12`。

p22–24 **网络互联**

原文翻译 (p22)

这些协议可以在铜缆或光纤上实现。噪声和阻抗损耗意味着比特率越高，电缆的最大长度越短。延迟通常在 100 ns 量级。

名称与年份	带宽
10BASE 以太网 (1990)	10 Mbps
100BASE 以太网 (1995)	100 Mbps
1000BASE 以太网 (1999)	1 Gbps
Myrinet-10G (2005)	10 Gbps
10GBASE 以太网 (2006)	10 Gbit/s
Infiniband 12X QDR (2009)	96 Gbit/s
100GBASE 以太网 (2010)	100 Gbit/s

原文翻译 (p23)

问题：假设我们把一个计算任务分配给了 N 个不同的节点， N 很大。在计算的某个时刻，某些数据需要在特定节点之间交换或共享。**如何只在需要它的那些节点之间高效地共享/交换必要的数据？**

→ 设计一个高效的互联结构、能在节点间共享数据，是一个高度非平凡的问题。大多数顶尖方案都受商业专利和专用软件的保护。

原文翻译 (p24)

注意这里是**微秒级**的延迟——1 微秒相当于 Nehalem CPU 上约 2000 个时钟周期。带宽也远低于 RAM 或缓存的带宽。

因此，**数据依赖很多的内存密集型计算往往难以在集群架构上运行。**把它们跑在单个节点上，可能比在多个节点上并行快得多。

讲解 · 这三页在讲同一件事：并行不是免费的

把 p24 那句话翻译成人话：**如果你的计算里各部分需要频繁交换数据，那么“分给更多节点”反而会更慢。**

粗略地看一个例子：假设一步计算要 $1\ \mu\text{s}$ ，而每步之后要跟别的节点通信一次，通信延迟也是 $1\ \mu\text{s}$ 。

- 单节点跑 n 步： $n \times 1\ \mu\text{s} = n\ \mu\text{s}$
- 2 节点各跑 $n/2$ 步，但每步多付 $1\ \mu\text{s}$ 通信： $(n/2) \times (1 + 1)\ \mu\text{s} = n\ \mu\text{s}$ 。一点没快。

通信开销不随节点数下降，而计算量随节点数下降——所以总有一个点，再加节点就开始变慢。

拓展 · Amdahl 定律（并行的天花板）

这一页背后的正式定律。设你的程序里有比例 p 的部分可以并行，剩下 $(1-p)$ 必须串行，用 n 个核，加速比是：

Amdahl 定律

$$S(n) = 1 / [(1 - p) + p/n]$$

符号	名称	含义
$S(n)$	加速比	用 n 个核比用 1 个核快多少倍
n	核数	并行使用的处理器核心数
p	并行比例	程序中可以并行的部分占的时间比例（0 到 1）
$1 - p$	串行比例	无论多少核都必须一个一个做的部分

致命的推论：令 $n \rightarrow \infty$ ，得到 $S_{\max} = 1/(1-p)$ 。

即使 95% 的代码能完美并行，用无限多个核，最多也只能快 20 倍。而 SCF 里的 Fock 矩阵对角化就是那个难并行的部分。这就是为什么 p2 说 Gaussian "DFT 也只能扩展到 32 核"。

原文翻译

名称	延迟	带宽	典型用途
RAM disk	纳秒	10 GB/s	一段被伪装成文件系统的 RAM，不常用
scratch / swap	微秒到毫秒	0.1–1 GB/s	专用文件系统，通常很快，用于存放 中间计算结果
local filesystem	毫秒	0.1 GB/s	存放操作系统文件、可执行程序、系统日志等
network filesystem	毫秒	0.1 GB/s	用户主目录，位于通过网络访问的外部硬件上
backup filesystem	秒	0.1 GB/s	不常访问的增量备份区

大量的输入输出（IO）操作会显著拖慢集群性能。请按集群管理员的指示，用 "scratch" 文件夹来运行计算。

这是作业里必须做对的一件事

你的 \$HOME 是**网络文件系统**——它挂在另一台机器上，所有节点通过网络访问它。而 Gaussian 一次计算会往硬盘写大量临时文件（.rwf、.int、.d2e），几十 GB 很常见。

如果你在 \$HOME 里跑，这几十 GB 的读写全部走网络，不但你的作业慢很多倍，还会拖累整个集群的文件系统，全系的人都受影响。

正确做法：在提交脚本里把工作目录设成本地 scratch。典型写法：

```
export GAUSS_SCRDIR=/scratch/$USER/$PBS_JOBID
mkdir -p $GAUSS_SCRDIR
cd $GAUSS_SCRDIR
# ...跑计算...
cp *.log *.chk $PBS_O_WORKDIR/      # 算完把结果拷回 $HOME
rm -rf $GAUSS_SCRDIR                # 清理 scratch
```

拓展 · 各种“盘”的名字对应关系

\$HOME	你的主目录，有配额（几十 GB），网络挂载，有备份。放代码、输入文件、最终结果。
---------------	--

scratch	临时高速盘，容量大（TB 级），没有备份，会被定期自动清空（通常 7–30 天）。放中间文件。
swap	严格说不是给用户的：当 RAM 不够时，操作系统把内存内容临时倒到硬盘上。一旦你的计算开始用 swap，速度会掉两个数量级——所以 %Mem 一定要设得小于节点实际可用 RAM。

p26 基本运算的标度 (BLAS 与 LAPACK)

原文翻译

矩阵分解（SVD、LU、对角化、Householder 等）非常昂贵、难以并行，并且无法从稀疏存储中获益。

运算	CPU 开销 / FLOPs	内存开销	稀疏时开销	并行执行
矩阵×矩阵	$O(n^3)$	$O(\text{nnz})$	更低	容易
矩阵×向量	$O(n^2)$	$O(\text{nnz})$	更低	容易
exp(矩阵)×向量	$O(n^2)$	$O(\text{nnz})$	更低	容易
矩阵求指数	$O(n^3)$	$O(\text{nnz})$	更低	容易
大多数矩阵分解	$O(n^3)$	$O(n^3)$	相同	困难

公式解析 · 表里的符号是什么意思

符号	名称	含义
n	矩阵维度	矩阵是 $n \times n$ 的。在量化里 $n \approx$ 基函数个数 N_{basis}
$O(\cdot)$	大 O 记号	"渐近标度"。 $O(n^3)$ 意思是： n 翻倍，时间变 8 倍。它只关心增长趋势，不关心前面的常数因子。
nnz	非零元个数	number of non-zeros。稀疏矩阵里绝大部分元素是 0，只需存 nnz 个数而不是 n^2 个。
FLOPs	浮点运算次数	加、减、乘、除各算一次。注意区分 FLOPs（次数）和 FLOPS（每秒次数）。

讲解 · 最后一行为什么是灾难

前四行都很友好：矩阵乘法虽然是 $O(n^3)$ 的计算量，但**内存只要 $O(\text{nnz})$** （稀疏时很小）、**而且极易并行**（把矩阵切成小块，各块独立算）。

但矩阵分解（对角化就是其中之一）三样全占：

- **内存也是 $O(n^3)$** ：因为分解过程中会产生大量新的非零元（"fill-in 填入"）——原本稀疏的矩阵，分解到一半就变稠密了。
- **稀疏没用**：同上，稀疏结构在分解过程中被破坏。
- **难并行**：数据依赖是全局的（见 p18 的解释）。

这一页是第 3 讲 SCF 循环的技术底座

去看第 3 讲 p60 的 SCF 流程图：**每一轮迭代都要"对角化 Fock 矩阵"一次**。典型的 SCF 要迭代 10–30 轮。所以对角化的开销要乘以 30。

这就是为什么"基组一大就慢得离谱"： N 从 100 增到 200，对角化时间 $\times 8$ ，双电子积分数量 $\times 16$ 。

拓展 · BLAS 和 LAPACK 是什么

- **BLAS (Basic Linear Algebra Subprograms)**：底层线性代数例程的**标准接口**，分三级——Level 1（向量·向量， $O(n)$ ）、Level 2（矩阵·向量， $O(n^2)$ ）、Level 3（矩阵·矩阵， $O(n^3)$ ）。
- **LAPACK**：建立在 BLAS 之上的高级例程——解线性方程组、最小二乘、**本征值问题（对角化）**、各种分解。

为什么要有"标准接口"：因为 Intel (MKL)、AMD (AOCL)、开源 (OpenBLAS) 各自写了针对自家 CPU 极致优化的实现，但都遵守同一套函数名和参数约定。**于是 Gaussian/ORCA 只要链接不同的库，就能在不同硬件上跑出最优性能，源码一行不用改。**一个手写的矩阵乘法可能比 MKL 慢一到两个数量级——因为 MKL 用了向量指令 (AVX-512)、缓存分块、多线程。所以科学计算里"永远不要自己写矩阵乘法"。

p27 Linux 统治超算界

原文翻译

自 2018 年起，全球 TOP 500 超级计算机 100% 运行 Linux 操作系统。

讲解 · 为什么是 100%，而不是 90%

1. **免费且开源**：500 台机器 $\times$ 每台几万个节点，任何按核心收费的授权模式都是天文数字。更重要的是可以改内核——超算厂商都要为自己的网络硬件、文件系统写驱动。
2. **可以裁到极薄**：计算节点上不需要图形界面、不需要音频、不需要蓝牙。每个节点省下的内存和 CPU 中断，乘以几万个节点就很可观。

3. **科学软件的生态在这里**：MPI、编译器（Intel/GCC/NVIDIA）、调度器、所有量化软件包，主力开发平台都是 Linux。
4. **远程管理天生就在设计里**：ssh、脚本、批处理是 Unix 的原生思维方式。

拓展 · 你在 Windows 上怎么办

三个选项，推荐程度从高到低：

1. **WSL2 (Windows Subsystem for Linux)**：Windows 自带，装个 Ubuntu，就有了完整的 Linux 命令行，`ssh`、`scp`、`vi`、`grep`、`awk` 全都有，还能直接访问 Windows 的文件。做 p39-41 那些练习最省事的方式。安装：管理员权限打开 PowerShell，敲 `wsl --install`。
2. **PuTTY + WinSCP**：讲义推荐的传统方案。PuTTY 负责登录，WinSCP 负责拖文件。缺点是本地没有 Linux 环境，练不了本地命令。
3. **Windows 10/11 自带的 OpenSSH**：在 PowerShell 里直接 `ssh user@host` 就能用，不用装 PuTTY。

p28-38 实践环节 #1：环境搭建

原文翻译（要点汇总）

- **任务 1**：下载并安装 NUS VPN 软件，登录 NUS VPN (nusit.nus.edu.sg/eguides/)
- **任务 2**：申请 HPC@NUS 账号 (nusit.nus.edu.sg/hpc/get-an-hpc-account/)，然后尝试登录
- **任务 3**：通过 ssh 连接到 NUS HPC 服务器上的账号。Windows 用 PuTTY；Mac OSX 直接开 Terminal 用 ssh 命令
- **任务 5**：Windows 用户签署条款后可在自己笔记本上安装 Gaussian 和 GaussView；Mac 用户需要 Windows 模拟器。在 HPC@NUS 上无需安装即可使用（已为所有用户装好）
- **ORCA**：闭源，对学术界免费。可在 Windows 原生安装但无图形界面。ORCA 6 在 HPC@NUS 上作为 module 提供，无需安装
- **OpenMOLCAS**：免费开源（GitLab）；**VASP**：商业闭源软件
- **可视化软件**：Chemcraft（商业，150 天试用）、VMD（免费，分子动力学与生物大分子）、Multiwfn（免费，用计算出的波函数计算大量性质）、Avogadro（免费，分子可视化与编辑）、JMOL（免费）、MOLDEN（免费）、Material Studio（商业，NUS 可用但授权数不多）
- **工具软件**：PuTTY（SSH 连接）、FileZilla / WinSCP（文件传输）、Notepad++（查看编辑文本文件）
- **必读**：HPC 新用户入门指南；如何运行批处理作业

讲解 · 这些工具各自解决什么问题

工具	解决什么
VPN	HPC 只对校园网开放。在校外必须先连 VPN，让你的电脑“看起来在校园里”。

ssh	加密的远程登录。你在本地敲的每个字符加密后送到服务器，服务器的输出加密后送回来。
scp / FileZilla / WinSCP	文件传输。scp 是命令行版（走 ssh 通道），后两个是图形界面版。
Notepad++	专门为了处理 换行符问题 ——见下面的拓展。
GaussView / Avogadro / Chemcraft	画分子（生成初始坐标）、看结果（轨道、密度、振动模式）。
Multiwfn	波函数后处理的瑞士军刀：各种电荷分析、键级、ELF/LOL、约化密度梯度 (RDG) 弱相互作用分析等。中文文档非常详尽（ sobereva.com ）。

Windows 用户几乎一定会踩的坑：换行符

Windows 的文本文件行末是 `\r\n`（回车+换行，CRLF），Linux 是 `\n`（LF）。你在 Windows 上用记事本写一个 `.sh` 脚本传到 Linux，运行时会报莫名其妙的错：

```
$ ./script.sh
bash: ./script.sh: /bin/bash^M: bad interpreter: No such file or directory
```

那个 `^M` 就是多出来的 `\r` ——Linux 以为解释器的名字叫 "bash<回车>"。

三种解法：① 在 Linux 上跑 `dos2unix script.sh`（p12 明确提到了）；② Notepad++ 里 编辑 → 档案格式转换 → 转换为 UNIX 格式；③ 直接在 Linux 上用 vi 写。

原文翻译

1) 只用终端命令行完成：

- 新建一个文本文件并填入随机文本（约 1000 词，30+ 行，从网上复制粘贴）。可以用 `vi` 或 `nano`
- 新建一个文件夹并把文件移进去（用 `mkdir` 和 `mv`）
- 复制一份该文件（用 `cp`）
- 修改副本，删掉其中一个词（用 `vi` 或 `nano`）
- 比较两个文件（用 `diff`）
- 数一数文件里 "and" 出现了多少次（用 `cat`、`grep`、`wc` 三个命令拼接）：

```
cat file1 | grep ' and ' | wc -l
```

2) 续：

- 用 `scp` 把两个文件传到 HPC 服务器：`scp file1 user@server:~/`
- 用 `ssh` 连接到 HPC：`ssh user@server`
- 在 HPC 的 `$HOME` 里新建并修改一个新文件 `file3`
- 用 `scp` 或 `FileZilla` 把 `file3` 传回自己的笔记本
- 阅读作业提交（#PBS 系统）的资料并照着例子执行、重复

讲解 · 逐条把命令说清楚

命令	全称/来源	作用
<code>mkdir</code>	make directory	建文件夹。 <code>-p</code> 表示“连中间层一起建，已存在也不报错”
<code>mv</code>	move	移动，也用来 重命名 （Linux 里改名就是移动到新名字）
<code>cp</code>	copy	复制。复制文件夹要加 <code>-r</code> （recursive，递归）
<code>diff</code>	difference	逐行比较两个文件，只打印不同的行
<code>cat</code>	concatenate	把文件内容打印到屏幕（或送进管道）
<code>grep</code>	global regular expression print	只输出匹配某个模式的行 。这是你以后分析 log 文件用得最多的命令
<code>wc</code>	word count	数数。 <code>-l</code> 数行数， <code>-w</code> 数词数， <code>-c</code> 数字节数

公式解析 · 管道 | 到底做了什么

Unix 管道

```
cat file1 | grep ' and ' | wc -l
```

拆开看，数据是这样流动的：

1. `cat file1` → 把整个文件的内容吐出来（比如 40 行）
2. `|` → **管道符**：把左边命令的**标准输出**，接到右边命令的**标准输入**上。不经过磁盘，直接在内存里传
3. `grep ' and '` → 从这 40 行里，只留下包含 " and "（前后带空格）的行，比如剩 12 行
4. `|` → 这 12 行继续往右传
5. `wc -l` → 数一数收到了几行 → 打印 12

这条命令其实有个陷阱，值得想一想

它数的是**"包含 and 的行数"**，不是**"and 出现的次数"**。如果一行里有两个 "and"，只算 1。

要真正数出现次数，得用：`grep -o ' and ' file1 | wc -l`（`-o` = only-matching，每个匹配单独输出一行）。

另外 `' and '` 两边的空格是为了避免匹配到 "sand"、"android"、"standard"。更严谨的写法是 `grep -ow 'and'`（`-w` = word boundary，词边界）。

拓展 · 这个技能后面直接变成生产力

第 2 讲 p101 的作业里有这一行：

```
grep 'SCF Done' "$input"*.log > results.txt
```

意思是：从 20 多个 log 文件里，把每个文件中含 "SCF Done" 的那一行（即最终能量）全抓出来，写进一个文件。**这就是你做势能面扫描时收集数据的标准手法**——手工从 20 个几万行的 log 里找能量要很久；这条命令瞬间完成。

几个立刻能用的变体：

```
grep -c 'imaginary' job.log      # 数虚频个数(应该是 0)
grep 'Normal termination' *.log # 看哪些作业正常结束了
grep -A5 'Mulliken charges' job.log # 找到这行,并多打印后面 5 行 (After)
grep -B2 'Error' job.log        # 找到报错行,并打印前面 2 行 (Before)
```

tail -20 job.log

看最后 20 行(排错第一步)

grep 'SCF Done' job.log | tail -1

只要最后一次的 SCF 能量

p41

PBS 作业提交脚本详解

原文翻译

3) 续： c. 写一个生成文本文件的简单脚本（示例如下）； d. 提交它，检查 file4 是否被生成。

```
#!/bin/bash
#PBS -P test_script
#PBS -q parallel
#PBS -l select=1:ncpus=12:mem=2GB
#PBS -j oe
cd $PBS_O_WORKDIR
if [ -f file4 ]; then rm -rf file4; fi
echo 'some random text' > file4
echo 'PBS_O_WORKDIR      =' $PBS_O_WORKDIR >> file4
echo 'ncpus              =' $ncpus >> file4
exit
```

公式解析 · 逐行拆解这个脚本

这一行	叫什么	它到底做了什么
#!/bin/bash	shebang	文件的 第一个字节 必须是 <code>#!</code> 。它告诉内核："用 <code>/bin/bash</code> 来解释这个文件"。这就是为什么多一个 <code>\r</code> 会导致 "bad interpreter" 报错。
#PBS -P test_script	项目名	Project 。用于计费 and 统计。注意： #PBS 对 bash 来说只是注释（# 开头），但 PBS 调度器会专门扫描这些行读取配置 ——这是个巧妙的设计：同一个文件既是合法脚本，又是作业描述文件。
#PBS -q parallel	队列	queue 。集群把节点分成若干队列： <code>serial</code> （单核，排队快）、 <code>parallel</code> （多核）、 <code>gpu</code> 、 <code>short / long</code> （按时限分）。选错队列 = 白排队。

#PBS -l select=1: ncpus=12:mem=2GB	资源申请	list of resources. <code>select=1</code> = 要 1 个"节点块"; <code>ncpus=12</code> = 每块 12 核; <code>mem=2GB</code> = 每块 2 GB 内存。 超过申请的内存会被系统直接杀掉作业。
#PBS -j oe	合并输出	<code>join output and error</code> 。把标准输出 (stdout) 和标准错误 (stderr) 合并到一个文件里, 方便排错。不加的话会生成两个文件 <code>jobname.o12345</code> 和 <code>jobname.e12345</code> 。
cd \$PBS_O_WORKDIR	切目录	这一行不能省。 作业开始时, 当前目录默认是你的 <code>\$HOME</code> , 不是你提交作业的那个目录。 <code>\$PBS_O_WORKDIR</code> (O = Original) 是 PBS 自动设置的变量, 记录了你敲 <code>qsub</code> 时所在的目录。不 <code>cd</code> 过去, 程序就找不到你的输入文件。
if [-f file4]; then rm -rf file4; fi	条件判断	<code>[-f file4]</code> 测试 "file4 存在且是普通file"。存在就删掉——避免和上次运行的结果混在一起。
echo 'xxx' > file4	重定向	<code>></code> = 覆盖写入 (原内容清空); <code>>></code> = 追加写入。把 <code>>></code> 写成 <code>></code> 是最常见的手滑, 会把前面写的全冲掉。

拓展 · 一个真能用的 Gaussian 提交脚本

把上面的骨架填成实际能跑的样子:

```
#!/bin/bash
#PBS -N myjob                                # 作业名字(Name)
#PBS -P personal-$USER
#PBS -q parallel
#PBS -l select=1:ncpus=8:mem=16GB
#PBS -l walltime=24:00:00                    # 最长运行 24 小时,超时被杀
#PBS -j oe

cd $PBS_O_WORKDIR
source /app1/ebenv                            # NUS HPC 加载环境的固定写法
module load Gaussian/16.C.02-AVX2

export GAUSS_SCRDIR=/scratch/$USER/$PBS_JOBID
mkdir -p $GAUSS_SCRDIR

g16 < myjob.gjf > myjob.log

formchk myjob.chk myjob.fchk                  # 把二进制 chk 转成文本 fchk
rm -rf $GAUSS_SCRDIR
```

三处必须自治

`ncpus=8` 必须和 Gaussian 输入文件里的 `%NProcShared=8` 一致; `mem=16GB` 必须大于 `%Mem` (留 2-4 GB 给程序本身的开销, 比如 `%Mem=12GB`)。

不一致的后果: `%NProcShared` 大于 `ncpus` → 12 个线程挤在 8 个核上互相抢, 比不并行还慢; `%Mem` 大于 `mem` → 作业被调度器直接 kill, log 文件里什么错误信息都没有, 只是突然截断。

讲解 · 三个必备的 PBS 命令

<code>qsub script.sh</code>	提交作业, 返回一个 JOBID (如 <code>12345.wlm01</code>)
<code>qstat -u \$USER</code>	查看自己所有作业的状态。Q = 排队中(Queued), R = 运行中(Running), E = 正在结束(Exiting), H = 被挂起(Held)
<code>qdel 12345</code>	删除/取消作业

p42 作业 (不计分) : 环境变量

原文翻译

尽可能熟悉以下内容:

- **命令行** (vi、ssh、scp、qsub、cat、sed、awk、bash 脚本、正则表达式等)。你用得越多, 在终端里的日子越好过
- **Linux 系统** (环境变量、`$PATH`、`$LD_LIBRARY_PATH`、Linux 与 Windows 的文件换行符)
- **在 HPC 和 NSCC 上提交作业** (两者类似, 但有一些小差别要注意)
- **在这些服务器与你的笔记本之间传输文件**

讲解 · \$PATH 和 \$LD_LIBRARY_PATH (第 2 讲 p95-96 会再讲一遍, 这里先说透)

\$PATH : 操作系统去哪儿找"可执行程序"

当你敲 `orca` 时, 系统并不会搜遍整个硬盘。它只会依次检查 `$PATH` 里列出的那几个目录, 找到第一个叫 `orca` 的可执行文件就运行。找不到就报 `command not found`。

```
echo $PATH
# /usr/local/bin:/usr/bin:/bin:/usr/games      <- 冒号分隔的目录列表

export PATH=$HOME/software/orca_6:$PATH        <- 把 orca 目录加到最前面
which orca                                     <- 确认系统现在能找到它了
```

注意 `$PATH` 写在末尾的写法 新目录:`$PATH` ——意思是"新目录优先, 但保留原有的所有目录"。如果写成 `export PATH=新目录 (漏掉 :$PATH)`, 你会把系统所有基本命令都弄丢, 连 `ls` 都用不了。

\$LD_LIBRARY_PATH : 程序运行时去哪儿找"动态库"

现代程序不会把所有代码都编进自己的可执行文件, 而是运行时才去加载共享库 (`.so` 文件, 相当于 Windows 的 `.dll`)。 `$LD_LIBRARY_PATH` 就是告诉动态链接器额外去哪些目录找这些 `.so`。

```
ldd $ORCA/orca      <- 列出 orca 需要的所有库, 以及找到没有
# libmpi.so.40 => not found    <- 这就是问题所在

export LD_LIBRARY_PATH=$HOME/software/openmpi/lib:$LD_LIBRARY_PATH
ldd $ORCA/orca      <- 再查一遍, 应该全部指向具体路径了
```

典型症状: 程序一运行就报 `error while loading shared libraries: libXXX.so: cannot open shared object file`。讲义 p96 说得很准: "这里没什么问题出错了, 操作系统只是不知道去哪儿找需要的库。"

拓展 · `module` 命令: HPC 上的标准解法

手动设 `$PATH` 和 `$LD_LIBRARY_PATH` 又烦又容易出错, 而且不同软件可能要求冲突的库版本。HPC 上的标准方案是 **Environment Modules**:

```
module avail          # 看有哪些软件可用
module load Gaussian/16.C.02  # 加载(自动设好所有环境变量)
module list           # 看当前加载了什么
module unload Gaussian    # 卸载
module purge          # 全部清空
```

`module load` 干的事就是执行一个脚本，帮你把该软件需要的 `$PATH`、`$LD_LIBRARY_PATH`、以及各种专有变量（`$GAUSS_EXEDIR` 等）一次设好。所以提交脚本里的 `module load` 一行都不能省——因为计算节点上的环境是全新的，不会自动继承你在登录节点上临时设的那些变量。

本讲小结：把第 1 讲的线索接到后面三讲

第 1 讲讲了什么	它在后面哪里回来
精度阶梯 (p2)：MM → 半经验 → HF → DFT → MP2/CC → FCI	第 4 讲整讲。核心问题：HF 丢掉的"电子相关能"怎么补回来，代价多大
电子-电子排斥项 $1/r_{12}$ 是万恶之源 (p7)	第 2 讲 p56 (He 原子)、p61 (为什么必须近似)、p83 (J 与 K 积分)
存储层级：数据搬运比计算贵 (p19, p25)	第 3 讲 p62–63：双电子积分存不下 → conventional / direct / semi-direct SCF
矩阵分解 $O(n^3)$、难并行 (p26)	第 3 讲 p60：SCF 每轮都要对角化 Fock 矩阵，这是主要瓶颈之一
并行有天花板 (p23–24, Amdahl)	第 3 讲 p2、p6、p9：各软件"不能扩展到超过 32 核"的原因
摩尔定律 vs 高次幂标度 (p21)	第 4 讲 p40：从 $O(N^4)$ 到 $O(N!)$ 的完整标度表
Linux / PBS / <code>\$PATH</code> (p27–42)	第 2 讲 p92–101 会完整重复一遍，那里还有真实的 Gaussian 扫描脚本

期中考可能从这一讲出的概念题

- 解释 latency 与 bandwidth 的区别，并说明为什么内存密集型计算在集群上并行反而可能更慢
- 为什么科学计算必须使用双精度浮点数
- 为什么矩阵对角化难以并行，而矩阵乘法容易
- 讲义列出的"未来方向"各自在解决什么缺陷
- 列出四个软件包各自的强项与适用体系

CM5235 复习解析·第 1 讲 | 源文件: CM5235_Lecture_01.pdf (42 页)

下一讲: [第 2 讲 量子力学基础与 Hartree-Fock 理论](#)